

1. **GitHub Repository Link:** <https://github.com/andresalonga/IT342-Salonga-AssetFlow.git>

2. Backend Project Naming Convention (Spring Boot)

The screenshot shows the Spring Initializr web application in a browser. The URL is start.spring.io. The application is configured for Spring Boot 3.5.11. The Project Metadata section shows the following details:

- Group: edu.cit.salonga
- Artifact: assetflow
- Name: assetflow
- Description: IT342-AssetFlow-Salonga
- Package name: edu.cit.salonga.assetflow
- Packaging: Jar (selected)
- Configuration: Properties (selected)

At the bottom, there are buttons for "GENERATE" (CTRL + G), "EXPLORE" (CTRL + SPACE), and a menu button.

3. Final Commit for Phase 1

IT342 Phase 1 – User Registration and Login Completed:

<https://github.com/andresalonga/IT342-Salonga-AssetFlow/commit/20f744663bccece98f0cc766ee49653ef1efdc6f>

The screenshot shows the GitHub commit page for the repository `andresalonga / IT342-Salonga-AssetFlow`. The commit is titled "IT342 Phase 1 - User Registration and Login Completed" and was committed 16 minutes ago by `andresalonga`. The commit hash is `20f7446`. The commit message is "IT342 Phase 1 - User Registration and Login Completed". The commit is on the `main` branch. The commit details show 98 files changed, with 14504 additions and 9 deletions. The commit is linked to the parent commit `c4dc1b2`. The commit details also show a search bar and a customizable line height option.

4. Screenshots of Implementation

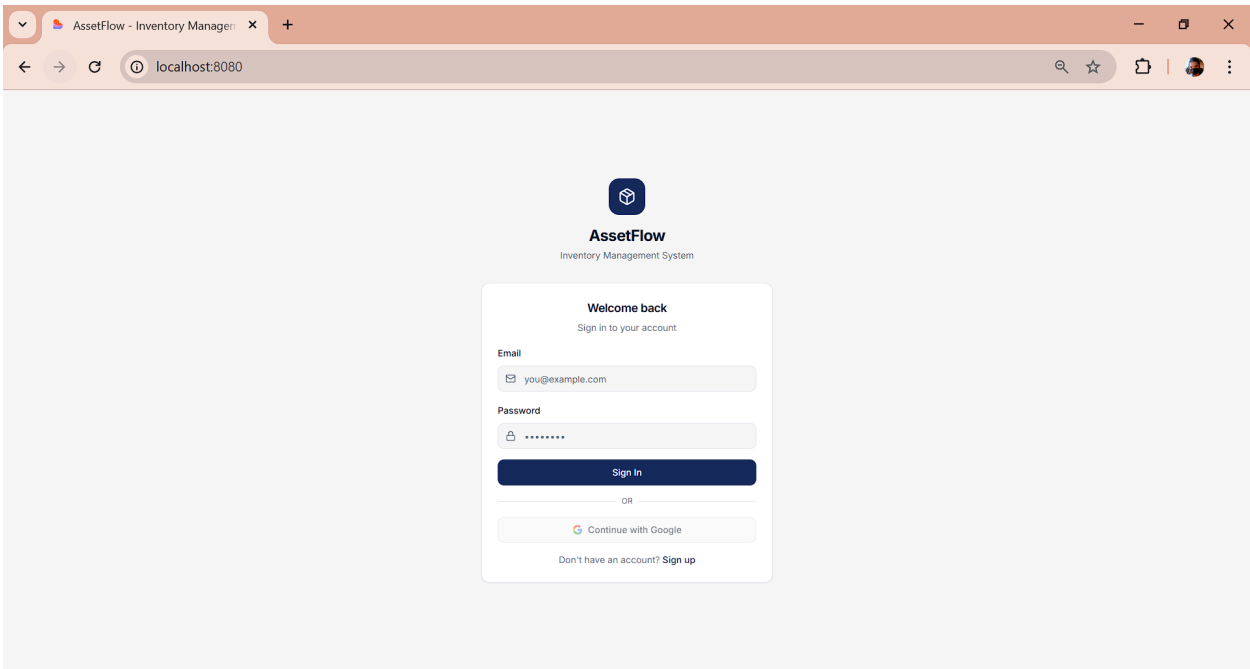
Registration page

The screenshot shows a web browser window with the title 'AssetFlow - Inventory Manager' and the address bar displaying 'localhost:8080/register'. The page features a dark blue logo at the top center, followed by the heading 'Create Account' and the subtext 'Join AssetFlow today'. Below this is a registration form with the following fields: 'First Name' (containing 'Maria'), 'Last Name' (containing 'Santos'), 'Email' (containing 'you@example.com'), 'Password' (masked with dots), and 'Confirm Password' (masked with dots). A dark blue 'Create Account' button is positioned below the form. At the bottom of the form, there is a link that says 'Already have an account? Sign in'.

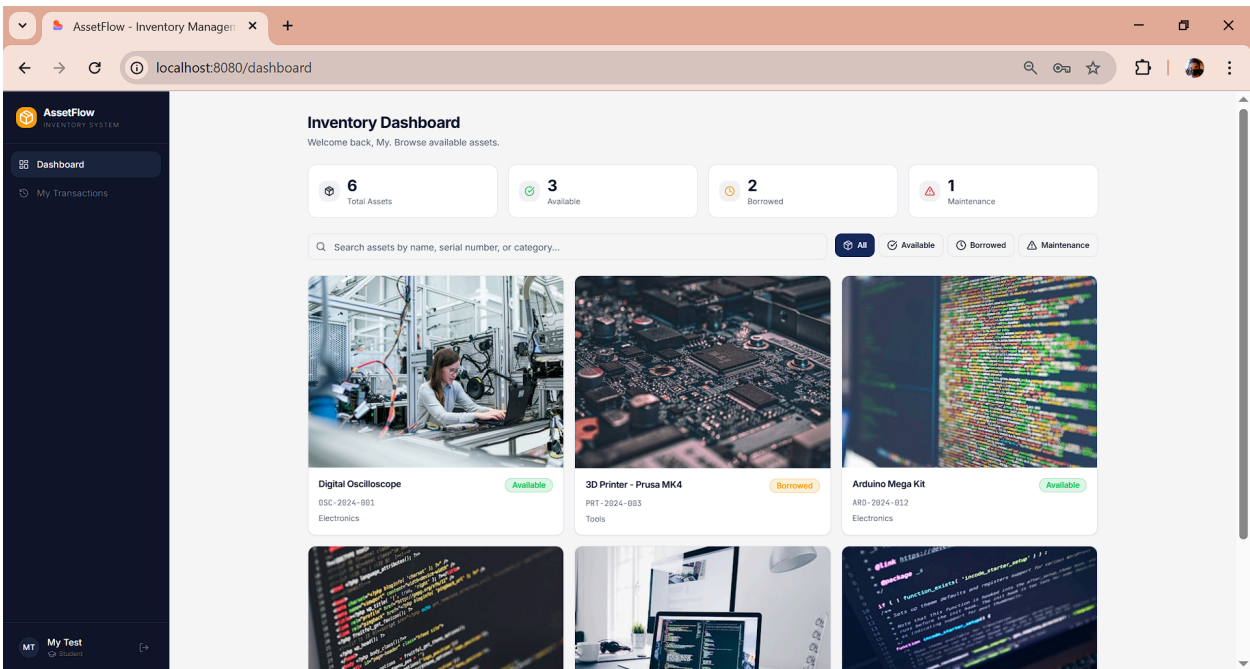
Successful user registration

This screenshot shows the same registration page as above, but with a green success message at the top of the form: 'Account created successfully! Redirecting to login...'. The form fields now contain test data: 'First Name' is 'My', 'Last Name' is 'Test', and 'Email' is 'my@test.com'. The 'Password' and 'Confirm Password' fields are still masked. The 'Create Account' button remains visible. Additionally, a green notification box in the bottom right corner of the page displays the message: 'Account created! You can now sign in with your credentials.'

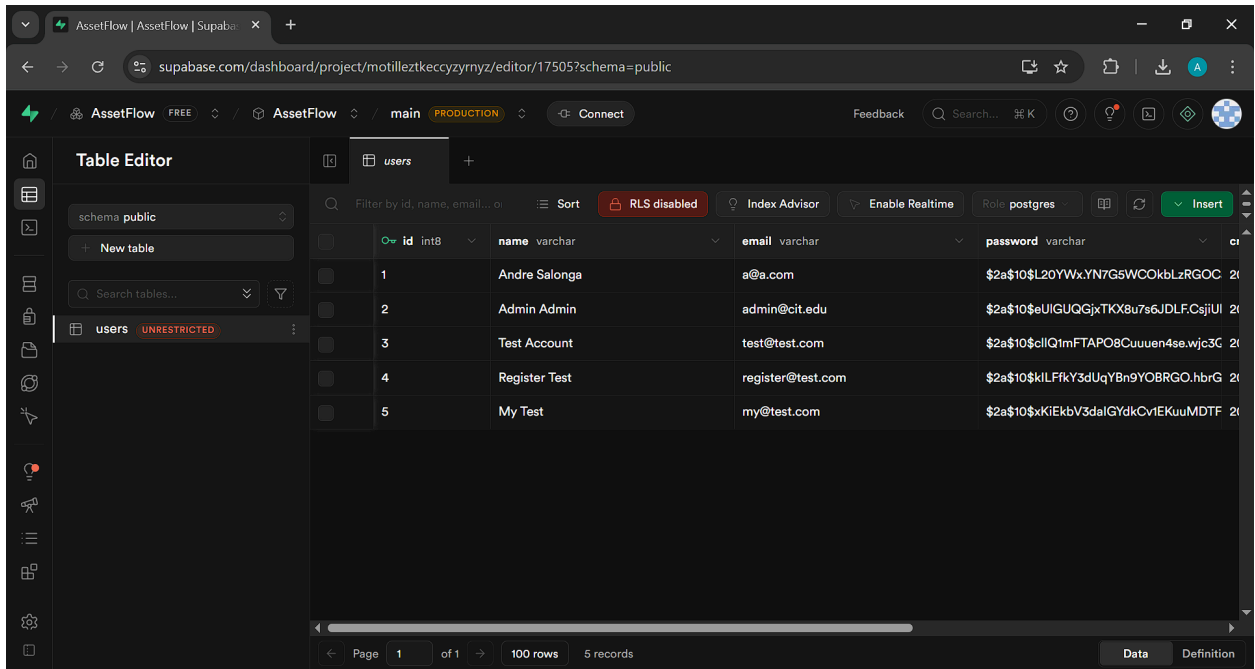
Login page



Successful login



Database record showing the registered user



The screenshot shows the Supabase Table Editor interface for the 'users' table in the 'public' schema. The table has 5 records. The columns are: id (int8), name (varchar), email (varchar), and password (varchar). The records are:

id	name	email	password
1	Andre Salonga	a@a.com	\$2a\$10\$L20YWx.YN7G5WCOkbLzRGOC 2a
2	Admin Admin	admin@cit.edu	\$2a\$10\$eUIGUQGjxTKX8u7s6JDLFCsjlU 2a
3	Test Account	test@test.com	\$2a\$10\$cllQ1mFTAPO8Cuuuen4se.wjc3G 2a
4	Register Test	register@test.com	\$2a\$10\$kILFfkY3dUqYBn9YOB8RGO.hbrG 2a
5	My Test	my@test.com	\$2a\$10\$xKIEkbV3dalGYdkCvtEKuuMDTF 2a

5. Short Implementation Summary

User Registration

Registration Fields Used:

Field	Type	Validation
name	String	Required, Not blank
email	String	Required, Valid email format
password	String	Required, minimum 6 characters

The frontend combines firstName and lastName into a single name field before sending to the backend.

Validation Process:

- **Frontend (RegisterPage.tsx):**
 - All fields are required (checked via `Object.values(form).some(v => !v)`)
 - Password and confirm password must match
 - Password must be at least 6 characters
 - Email format validated via HTML5 input type
- **Backend (RegisterRequest.java):**
 - Uses Jakarta Bean Validation (`@NotBlank`, `@Email`, `@Size`)
 - Server-side validation ensures data integrity even if client validation is bypassed

Duplicate Account Prevention:

- The `UserRepository.existsByEmail()` method checks if the email already exists before registration
- The `AuthService.register()` method returns an error response: "Email already registered" if a duplicate is found
- The email column in the database has a unique constraint (`@Column(unique = true)` in `User.java`)

Secure Password Storage:

- Passwords are hashed using BCrypt (via `BCryptPasswordEncoder`)
- The raw password is never stored in the database
- BCrypt includes automatic salt generation for each password

User Login

Login Credentials Used:

Field	Type	Validation

email	String	Required, Valid email format
password	String	Required

Verification Process:

1. Frontend sends email and password to POST /api/auth/login
2. AuthService.login() retrieves the user by email from the database
3. If user not found → returns "User not found" error
4. If found → uses encoder.matches() to compare the plaintext password against the stored BCrypt hash
5. If password doesn't match → returns "Invalid credentials" error

Post-Login Behavior:

1. On successful authentication, a JWT (JSON Web Token) is generated via JwtUtil.generateToken()
2. The token is returned to the client along with user info
3. Frontend stores the token in localStorage via saveToken()
4. User data is stored in localStorage for session persistence
5. User is redirected to /dashboard
6. The JWT token is included in subsequent API requests via the Authorization: Bearer <token> header

Database Table

Table: users

Column	Type	Constraints
--------	------	-------------

id	BIGINT	Primary Key, Auto-increment
name	VARCHAR(255)	Not null
email	VARCHAR(255)	Unique, Not null
password	VARCHAR(255)	Not null (BCrypt hash)
role	VARCHAR(255)	Default: 'USER'
created_at	DATETIME	Auto-generated

API Endpoints

Method	Endpoint	Description
--------	----------	-------------

POST	/api/auth/register	Register new user
POST	/api/auth/login	Authenticate user, get JWT
GET	/api/auth/me	Get current authenticated user
POST	/api/auth/logout	Logout (client removes token)